Generative AI Systems Design interview

101 subscribers

1. Introduction and Overview
2. Ad Ranking
3. Ad Retrieval
4. Text-to-Video Generation
5. ChatGPT: Focused Transformer
6. Image Captioning
7. Multimodal Augmented Generation
8. Parallelism
9. High Bandwidth Image Synthesis
10. Text to Image Generation
11. Personalized Recommendation
12. Text-to-Video Generation

11 Text-to-Video Generation

Introduction

Text-to-video generation is a key application of generative AI, enabling the generation of video from textual descriptions. This chapter delves into the intricacies required to scale such a system for real-world applications.

Figure 1: An overview of text-to-video generation using a text encoder and a video generation model.

Clarifying Requirements

Here's a typical interview between a candidate and an interviewer.

Candidate: What is the primary business objective for the generated videos?
Interviewer: We want to offer this second-only videos.
Candidate: What are the expected output specifications?
Interviewer: We should aim for high-definition quality, to ensure the videos look a wide range of modern platforms and devices. Let's aim for 720p resolution.
Candidate: Is 24 frames per second (FPS) the standard rate for the generated video?
Interviewer: Yes.
Candidate: What is the expected latency for generating a video?
Interviewer: While we want to be competitive with existing services, the main focus is on the efficiency of innovating time will be acceptable in future iterations, we will optimize for efficiency and speed.
Candidate: Should we focus on a specific video category?
Interviewer: Yes, the system should generate videos across various genres and subjects.
Candidate: Should the system support fine-tuning or LoRA adapters for style or character consistency?
Interviewer: Yes, that's a crucial feature. The system should support fine-tuning.
Candidate: What are the expected video includes audio output?
Interviewer: For the initial version, let's focus on high-quality silent video. Adding synchronized audio is a powerful feature, but it's a priority at this stage.
Candidate: What is the expected scale of the service?
Interviewer: We have a large video dataset, around 100 million diverse videos with captions. Some captions might be noisy or incomplete.
Candidate: What is the expected latency for building a model video to create a realistic a pretrained text-to-image model to familiar videos. Do we have a pre-trained text-to-image model?
Interviewer: Yes, that is a fair assumption.
Candidate: Considering the high computational demands of video generation, what is our compute budget?
Interviewer: We have a substantial budget, but we're mindful of providing significant computational resources. We have over 4000 H100 GPUs (32 available for real-time inference.
Candidate: Great. We should the system has safeguards to prevent generating offensive or harmful videos is crucial?
Interviewer: Great point. Yes, we need to ensure our proposed system is safe and responsible.

Frame the Problem as an ML Task

The candidate then proceeds to define the problem as a machine learning task and highlights the necessary considerations beyond those found in Chapter 6 for text-to-image generation.

Specifying the system's input and output

The input is a descriptive text conditioning a scene, action, or narrative. The output is a flow-second 720p (1280 x720) video that accurately reflects the input text, maintaining coherence across frames.

For example, given a text input like "A dog playing fetch in a park on a sunny day," the system should generate a video depicting the scene, capturing the dog's movements, the park's environment, and the atmosphere of a sunny day.

Figure 2: Input and output of a text-to-video system.

Choosing a suitable ML approach

Text-to-video generation is a complex task, demanding a state-of-the-art approach. Both generative models from textual descriptions. Techniques such as autoregressive modeling and diffusion models that are adapted to handle the added dimension of time. The system must also incorporate mechanisms for ensuring temporal consistency, maintaining coherence in objects and characters across frames, and avoiding artifacts and unrealistic visuals. Therefore, we choose diffusion models to develop our text-to-video generation system.

There, however, a crucial difference between them. For video generation, the model must process and generate a sequence of frames rather than a single image. This significantly increases the computational load. For instance, generating a 5-second video at 24 FPS means the model must produce 120 frames. A 10-second 1080p image takes around 1 second to generate on a high-end GPU, but a naive approach to 10-second 720p video could take 120 seconds. Optimizing the model to generate frames less dense (8 frames-long instead of 120 frames, for example), generating from second 720p video could take around minutes.

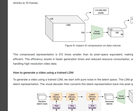

Figure 3. Text-to-video generation requires more computation than text-to-image generation.

To address this complexity, we propose a cascaded diffusion approach, a common technique for high-resolution image generation. This approach uses first a pre-trained text-to-image diffusion model (Imagen, Dall-E) and then applied and used by most video generation models, such as Imagen Video, Cascade Video Diffusion, Dall-E 2, and Imagen Video. Our system's architecture is illustrated below.

Latent diffusion model (LDM)

To address the complexity and computational cost of video generation, we employ latent diffusion models (LDM) approach. This approach uses first proposed by the Stable Diffusion paper, LD and later applied and used by most video generation models, such as Imagen Video, Cascade Video Diffusion, Dall-E 2, and Imagen Video. Our system's architecture is illustrated below.

Figure 4. Latent diffusion models significantly reduce computational cost compared to pixel-space diffusion models.

LDM makes primarily on a compression network to compress video frames into a lower-dimensional space, significantly reduces the computational resource to move detail.

Figure 5. Latent diffusion models significantly reduce computational cost compared to pixel-space diffusion models.

The compression representation is then used to create frame by pixel-space diffusion models, making LDM training 100 times less stress efficient. The efficiency makes training feasible on a cluster of 32 available GPUs. The compression network is usually based on a variational autoencoder (VAE) (Q) model that is trained separately from the diffusion model. The encoder (E) compresses the input video into a lower-dimensional representation, which is then used as the ground truth for the diffusion process. The decoder (D) reconstructs the denoised representation back into a pixel-space video.

Figure 6. Diagram of compression on video stress.

How does LDM address computational complexity?

LDMs require less computing power than standard diffusion because processing compressed representations is cheaper than processing high-dimensional pixel data. The computational cost of the diffusion process is reduced significantly.

Imagine we have a video with 100 frames, each 1080p in resolution and 24 bits per pixel (3 bytes). This equals 622 megabytes (MB) of data. A 5-second video at 24 FPS is a dataset of 3 GB. Working with such large datasets is inefficient. A VAE might compress this video into a representation of, for example, 100 megabytes (MB). This reduction in size makes the network and system resources by a factor of 6 for video's pixelar dimension resources. BM16 pixels, and its temporal dimension density to 10 frames.

Figure 7. Video generation using a trained LDM.

For the chapter, we choose an LDM approach to develop our text-to-video generation system because it's efficient and it reduces computational cost. Now let's move to the data preparation.

Data Preparation

The system requires a large dataset of text-video pairs to function effectively. We utilize a combination of public and internal datasets. These pairs cover various subjects and actions, allowing the model to learn diverse visual concepts. In this section, we prepare the videos and captions for training our LDM.

Video preparation

We focus on the steps involved in preparing videos for training:

- The raw video dataset
- Frame extraction
- Frame-level data preprocessing

Filter inappropriate videos

Large datasets often contain unwanted content. This step removes inappropriate videos to ensure the model learns only high-quality, safe data. Common steps include:

- **Remove low-quality or corrupted videos:** We filter videos that are corrupt, have excessive resolution, show static, or demonstrate videos with compression artifacts.
- **Remove inappropriate or harmful content:** We use a classification method such as CV to eliminate denoted videos. This ensures training data is diverse and the model will not be exposed to certain types of harmful content.
- **Remove harmful content:** We use them detection models to identify and remove videos with explicit content. This step is vital for ensuring the model does not generate harmful or inappropriate content.

Standardize videos

- **Adjust video length:** We split longer videos into five-second clips to ensure training data consists only of videos of the same length.
- **Standardize frame rate:** We re-encode the videos with higher frame rates to 24FPS to ensure all the videos have the same temporal resolution.
- **Adjust video dimensions:** We resize and crop videos to a standard size, for example, 1280x720 pixels.

Preprocess video representations in the latent space

As the LDM operates in the latent space, it needs video representations as input. Thus, each training iteration outputs require the following steps:

1. Extract frames from a video in the pixel space.
2. Feed the extracted frames into the compression network to obtain latent representations.
3. Use the latent representations to construct noisy-the diffusion model.

However, these steps are sufficiently time-consuming and computing-heavy for millions of videos each time we train a text-to-model video diffusion training. Computing latent representations on the fly is resource intensive and time-consuming.

To optimize the process, we precompute the latent representations for all videos and save them in storage. During training, the diffusion model directly accesses the precomputed latent representations without waiting for frame extraction or compression. This significantly speeds up the training process. The trade-off is increased storage requirements, but the benefits of faster and manageable data loading outweigh the storage cost.

Figure 8: Text-to-video training system using latent for training.

Caption preparation

It's important to have high-quality, consistent captions. Some captions are likely to be missing or irrelevant. Common steps for preparing captions are:

- **Handle missing or non-English captions:** For videos without captions or with captions in another language, we use models to generate English captions. This ensures all videos have usable, English-language captions.
- **Re-captioning:** We use a vision-language captioning model (similar video capturing models such as pelican) trained on a LLM generates longer, more detailed descriptions. The focus here CLIP has shown that this process is essential for enhancing quality and alignment.
- **Caption standardization:** We use a language model to standardize captions, ensuring consistency in capitalization, punctuation, and style. We encode its precomputes caption embeddings, leveraging an CLIP training.

Figure 9: Video data preprocess.

Model Development

Architecture

While cutting-edge the architecture for text-to-video diffusion models, we have two main options: a U-Net and DiT. We'll examine both and determine the additional layers required to extend them to handle videos.

U-Net for videos

U-Net is a widely used U-Net architecture before embedding it to process videos, we first adapt it to operate on 2-frame. By a U-Net operates on 2-frame, it can learn to denoise individual frames while maintaining spatial coherence. The 2-frame U-Net processes each frame independently, ensuring that the model learns to generate high-frame-rate a cross-frame attention layer to ensure consistent 360 3D frame-to-frame. The next primer.

Figure 10: U-Net architecture for image generation.

However, these layers mainly focus on capturing the relationships between pixels within a single image. The design presents a challenge when dealing with video data, where understanding the relationships between frames is crucial for ensuring smooth and realistic motion. Thus, temporal layers are added to the architecture to address this issue.

To address this shortcoming, we modify the U-Net architecture to understand relationships across frames. In particular, our board has commonly used temporal layers:

- Temporal attention
- Temporal convolution

Figure 11: U-Net architecture for image generation.

However, these layers mainly focus on capturing the relationships between pixels within a single image. The design presents a challenge when dealing with video data, where understanding the relationships between frames is crucial for ensuring smooth and realistic motion. Thus, temporal layers are added to the architecture to address this issue.

To address this shortcoming, we modify the U-Net architecture to understand relationships across frames. In particular, our board has commonly used temporal layers:

- Temporal attention
- Temporal convolution

Figure 12: Temporal layers: attention layer and convolution layer.

- **Temporal convolution:** Temporal convolution refers to applying a convolution operator in a 3D segment of data, to capture the temporal dimension. Figure 13 illustrates 3D and 1D temporal convolution.

Figure 13: Convolution in 3D, convolutions.

In summary, to extend a U-Net architecture to process videos, we use three-dimensional convolution and temporal attention layers in each downsampling and upsampling block. These layers allow the U-Net architecture to model the relation in input videos and generate a sequence of frames that are temporally consistent. To learn more about how these layers can be implemented, refer to Imagen Video.

DiT for videos

Unlike U-Net, which is based mainly on convolutions, DiT relies primarily on the Transformer architecture. Its vision in Figure 14, the backbone of Sora video compression.

Figure 14: Diagram of Sora video compression.

Let's examine each component and understand its purpose:

Patchify

This component converts the input to a sequence of embedding vectors. It first divides the input into smaller, fixed-size patches. Each patch is then flattened and projected onto a vector. The flattened patches are then transformed into patch embeddings. This process effectively converts the visual data into a format suitable for the subsequent Transformer layers. Let's look at Patchifying in more detail.

The patchify process is similar for both image and video inputs. For images, it divides the input into fixed-size 2D patches. For videos, it takes a tubelet from the input.

Figure 15: Patchify for image, in video.

Positional encoding

The positional encoding component produces an embedding for each position in the original sequence. These embeddings provide the Transformer with information about the relative or absolute position of each patch.

Figure 16: Positional encoding for image, in video.

As we saw in Chapter 7, there are different ways to encode positions. Sora's transformer uses fixed positional encoding during training, while other methods make the positional embeddings learnable. There are also three ways to assign positions to each patch. For example, we can give each patch a single number for show its place in a sequence or use 3D coordinates (2D for images) to show where each patch is in space and time.

Figure 17: 1D, 2D, and 3D positional encoding.

There is not one best way to do positional encoding. We often need to run experiments to find the approach that will be most effective for the task and data. In the case of Sora, the transformer processes up to 2048 (16x16x32) patches embedding. To learn more about positional encoding in text-to-video models, refer to [4].

Transformer

The Transformer processes the sequence of embeddings and other conditioning signals, such as the text prompt, to predict noise for each patch.

Figure 18: Transformer architecture.

Unpatchify

Unpatchify converts the patchelated noise latent back to the original input dimensions. It includes a pseudo-inv for normalization or a linear projection layer, and reshaping patches back to form the final output.

Figure 19: Unpatchify compression.

U-Net vs. DiT

Both U-Net and DiT architectures are proven to be effective for text-to-video generation. The U-Net architecture has been around for longer and has been extensively tested. Popular U-Net-based text-to-video models include Stable Video Diffusion [5] by Stability AI and Emu video [6] by Meta.

The DiT architecture is more recent and has shown great promise, with superior results. DiT performs better with increased data and computational resources, demonstrating its scalability. The Transformer's ability to model long-range dependencies is particularly valuable in video data, as it can capture relationships between frames effectively. Notable models utilizing DiT include Sora and Open-Sora, which are popular open-source implementations of the DiT architecture.

In this chapter, we follow Sora and choose the DiT architecture.

Training

Training a video diffusion model is very similar to training an image diffusion model. During training, we add noise to the original video and ask the model to predict the added noise. The model learns to predict the noise by minimizing the difference between the actual noise and the predicted noise.

Training a video diffusion model is very similar to training an image diffusion model. During training, we add noise to the original video and ask the model to predict the added noise. The model learns to predict the noise by minimizing the difference between the actual noise and the predicted noise.

MSE objective and loss function

The loss function quantifies the difference between noise, calculated using the mean squared error (MSE) formula. This loss measures the difference between the predicted noise and the actual noise, encouraging the model to accurately predict the added noise. The objective is to minimize this difference, thereby improving the model's noise prediction accuracy.

Researchers have experimented with adding other loss functions to enhance text-to-video performance. To learn more, refer to [7].

Challenges in training video diffusion models

Training a DiT model for text-to-video generation requires significant computational and storage resources. This section explores two key challenges:

- Lack of large-scale video text data
- Computational cost of high-resolution video generation

Lack of large-scale video-text training data

Training large models requires vast amounts of data. While readily available for text-to-image models, where a huge amount of image-text pairs are available, paired video-text data is scarce. This scarcity presents a challenge in training effective video generation models.

There are two common strategies for addressing the lack of large-scale data:

1. **Pre-training on image data:** This strategy first pretrains the DiT model on image-text pairs to leverage extensive datasets and learn visual concepts from text descriptions. The pretrained model is then fine-tuned on video-text data for video generation.
2. **Precision the DiT needed on image data:** This strategy first pretrains the DiT model on image-text pairs to leverage extensive datasets and learn visual concepts from text descriptions. The pretrained model is then fine-tuned on video-text data for video generation.

Figure 20: Training on image-text data.

Figure 21: Training on image-text data.

Figure 20: Two strategies to utilize image–text training data

Both strategies leverage hundreds of millions of image–text data in training, allowing the DiT model to learn from both images and videos. For simplicity, we choose the first strategy, as it requires only one stage of training. However, both strategies can be effective in practice.

Computational cost of high-resolution video generation

As discussed earlier, processing and generating videos is more expensive than images. This is primarily because videos generally contain hundreds of frames, making the process slower and more costly. Generating high-resolution videos, such as 720p or 1080p, adds to the challenge.

Here are a few common strategies to reduce the computational cost of training high-resolution video generation models:

- **Employ an LDM-based approach:** Instead of training the DiT model directly in pixel space, we use a compression network to convert videos from pixel space into a lower-dimensional latent space. Training the diffusion model in this latent space reduces the computational load.
- **Precompute video representations:** By precomputing video representations in the latent space before training, we avoid repetitive computations during training. Utilizing this method saves time and speeds up the training process.
- **Utilize a spatial super-resolution model:** As proposed by Google's "Imagen video" [15], we use a separately trained model to upscale the resolution of generated videos. The DiT model generates videos at a lower-resolution that are then enhanced to the desired resolution by a spatial super-resolution model. For example, the DiT model can generate videos at 720p, and a spatial super-resolution model can then upscale them to 1080p or 4K.
- **Utilize a temporal super-resolution model:** As proposed by [15], we employ a model to increase the temporal resolution by interpolating between frames. For instance, if a video should be five seconds at 24 FPS (i.e., 120 frames total), the DiT model can generate it at 12 FPS (60 frames), and a temporal super-resolution model can then interpolate to achieve 24 FPS.
- **Use more efficient architectures:** We can adopt an efficient implementation of the attention mechanism [16] to reduce the computational load during training. Additionally, techniques like Mixture of Experts (MoE) [17] can be used to accelerate the training process.
- **Use distributed training:** We use distributed training techniques, such as tensor parallelism, to parallelize training across multiple devices. By splitting the model, data, or both across different devices, we can significantly speed up training and handle larger video datasets more efficiently. This approach is particularly useful for high-resolution video generation, where memory and computational demands are substantial. For an overview of distributed training, refer to Chapter 1.

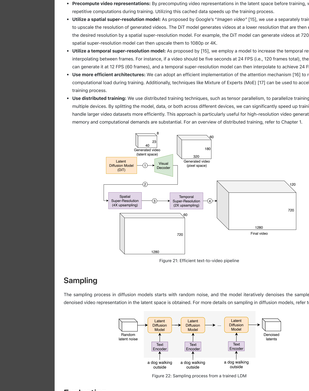

Figure 21: Efficient text-to-video framework

Sampling

The sampling process in diffusion models starts with random noise, and the model iteratively denoises the sample until a fully denoised video representation in the latent space is obtained. For more details on sampling in diffusion models, refer to Chapter 9.

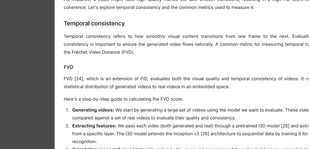

Figure 22: Sampling process from a trained LDM

Evaluation

Offline evaluation metrics

A consistent benchmark is crucial for evaluating video generation models. VBench [18] and *Movie Gen Bench* [19] offer this by providing a curated set of prompts designed to test various aspects of video generation such as motion coherence, temporal consistency, and scene complexity. We can use these benchmarks to measure how well the model creates realistic and smooth videos, focusing on video quality, motion accuracy, and scene transitions. Let's explore both automated metrics and human evaluation, focusing on three key areas:

- Frame quality
- Temporal consistency
- Video–text alignment

Frame quality

Frame quality refers to measuring the quality of each frame independently. To measure this quality we employ FID [20] and Inception score (IS) [21], both of which are commonly used for images. The overall quality is calculated by averaging the FID and IS scores of all frames. Other metrics such as LPIPS [22] and KID [23] can also be used.

While FID and IS measure the quality of individual frames, they don't account for the temporal consistency in generated videos. For instance, a video might have high-quality frames but lack smooth transitions, resulting in a high FID score without visual coherence. Let's explore temporal consistency and the common metrics used to measure it.

Temporal consistency

Temporal consistency refers to how smoothly visual content transitions from one frame to the next. Evaluating temporal consistency is important to ensure the generated video flows naturally. A common metric for measuring temporal consistency is the Fréchet Video Distance (FVD).

FVD

FVD [24], which is an extension of FID, evaluates both the visual quality and temporal consistency of videos. It compares the statistical distribution of generated videos to real videos in an embedded space.

Here's a step-by-step guide to calculating the FVD score:

1. **Generating videos:** We start by generating a large set of videos using the model we want to evaluate. These videos will be compared against a set of real videos to evaluate their quality and consistency.
2. **Extracting features:** We pass each video (both generated and real) through a pretrained I3D model [25] and extract features from a specific layer. The I3D model extends the Inception v3 [26] architecture to sequential data by training it for action recognition.
3. **Calculating mean and covariance:** We calculate the mean and covariance of the extracted features separately for generated and real videos. These statistical measures summarize the distribution of features for both sets of videos.
4. **Computing Fréchet distance:** We calculate the FVD score as the Fréchet distance between the mean and covariance of generated and real videos. The Fréchet distance measures how close the two distributions are.

A lower FVD score indicates greater similarity between the distributions, meaning the generated videos are more realistic and temporally consistent.

Video–text alignment

Video–text alignment refers to how accurately the generated video reflects the textual description on which it was conditioned.

A commonly used metric to measure video–text alignment is the CLIP similarity score, calculated as follows:

1. **Extracting frame-level features:** We pass each video frame through a pretrained CLIP image encoder to get visual features. The text is encoded using the text encoder to obtain textual features.
2. **Calculating similarities:** For each frame, we compute the cosine similarity between its visual features and the textual features. This score indicates how well the frame content aligns with the text.
3. **Aggregating per-frame similarities:** We aggregate these similarity scores to get a single score representing the overall video–text alignment. Aggregation can be done by averaging, taking the maximum score, or using other statistical methods.

A high CLIP similarity score indicates that generated videos are aligned with their corresponding text.

Human evaluation

Alongside the described automated metrics, human evaluation is still vital for assessing generative models, as it provides a subjective assessment that complements automated measures.

For human evaluation, we generate videos from text prompts using two different models. We then present pairs of videos, one from each model, to human annotators. They choose the better video based on assessing video–text alignment, video quality, and temporal consistency. This process allows us to compare two models to see which one performs better.

Online evaluation metrics

Online evaluation metrics for text-to-video models are similar to those for text-to-image models. Important metrics include:

- Click-through rate
- Time spent on the page
- User feedback
- Conversion rate

These metrics help gauge user engagement, satisfaction, and overall model performance in production.

Overall ML System Design

In this section, we dive into the holistic design of a text-to-video generation system. In particular, we examine the following pipelines:

- Data pipeline
- Training pipeline
- Inference pipeline

Data pipeline

The data pipeline prepares training data by filtering unsuitable images and videos, standardizing them, and precomputing and storing latent representations. It ensures captions are relevant and detailed by re-captioning and using a pretrained text encoder to precompute and store caption embeddings.

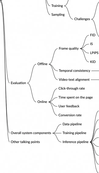

Figure 23: Data pipeline

Training pipeline

The training pipeline trains the model using the training data prepared by the data pipeline.

Inference pipeline

The inference pipeline processes real-time user requests to generate videos from text prompts. As shown in Figure 24, it has several crucial components that ensure system quality and safety.

Figure 24: Inference pipeline components

Most of the components are similar to those explored in Chapter 9 for text-to-image generation. The unique components for text-to-video generation are:

- Visual decoder
- Temporal super-resolution

Visual decoder

The LDM generates output in the latent space, not the pixel space. The visual decoder then uses the compression network to convert this latent representation back into the pixel space.

Temporal super-resolution

This component interpolates between the generated frames, leading to smoother motion in videos.

Other Talking Points

In case there's extra time at the end of the interview, you might discuss these further topics:

- Ensuring sampling flexibility for variable durations, resolutions, and aspect ratios [1]
- Extending the text-to-video model to downstream applications such as inpainting, outpainting, video-to-video stylization, frame interpolation, super-resolution, and animating images (image-to-video) [19]
- Support for controlling the generated videos such as the level of desired motion and the type of motion (camera vs. object motion) [27].
- Using progressive distillation techniques to reduce the computational demands of training [28].
- Details of spatial and temporal super-resolution models [16].
- Details of re-captioning model [9][29].
- Different noise schedulers. [29].
- Noise conditioning augmentation techniques [30].
- Personalizing a text-to-video model to a particular subject [31].
- Controlled for text-to-video models [32].
- Details of Stable Cascade method [33].
- Details of visual compression network [13].

Summary

Reference Material

[1] Video generation models as world simulators. https://openai.com/index/video-generation-models-as-world-simulators/
[2] H100 Tensor Core GPU. https://www.nvidia.com/en-us/data-center/h100/
[3] High-Resolution Image Synthesis with Latent Diffusion Models. https://arxiv.org/pdf/2112.10752
[4] Meta Movie Gen. https://ai.meta.com/research/movie-gen/
[5] Auto-Encoding Variational Bayes. https://arxiv.org/pdf/1312.6114
[6] The Illustrated Stable Diffusion. https://jalammar.github.io/illustrated-stable-diffusion/
[7] On the Do-qualification of LAION-2B. https://arxiv.org/abs/2303.12733
[8] The Llama 3 Herd of Models. https://arxiv.org/abs/2407.21783
[9] Llava Next(7.6 Strong Zero-shot Video Understanding Model. https://llava-vl.github.io/blog/2024-04-30-llava-next-video/
[10] Lumiere: A Space-Time Diffusion Model for Video Generation. https://arxiv.org/abs/2401.12945
[11] OpenSora Technical Report. https://github.com/hpcaitech/Open-Sora/blob/main/docs/report_03.md
[12] ReFormer: Enhanced Transformer with Rotary Position Embedding. https://arxiv.org/abs/2104.09864
[13] Stable Video Diffusion: Scaling Latent Video Diffusion Models to Large Datasets. https://arxiv.org/abs/2311.15127
[14] Ereo Video: Factorizing Text-to-Video Generation by Explicit Image Conditioning. https://arxiv.org/abs/2311.10709
[15] Imagen Video: High Definition Video Generation with Diffusion Models. https://arxiv.org/abs/2210.02303
[16] HyperAttention: Long-context Attention in Near-Linear Time. https://arxiv.org/abs/2310.05869
[17] Mixture of Experts Explained. https://huggingface.co/blog/moe
[18] VBench: Comprehensive Benchmark Suite for Video Generative Models. https://vchbench.github.io/VBench-project/
[19] Movie Gen Bench. https://github.com/facebookresearch/MovieGenBench
[20] FID calculation. https://en.wikipedia.org/wiki/Fr%C3%A9chet_inception_distance
[21] Inception score. https://en.wikipedia.org/wiki/Inception_score
[22] The Unreasonable Effectiveness of Deep Features as a Perceptual Metric. https://arxiv.org/abs/1801.03924
[23] Demystifying MMD GANs. https://arxiv.org/abs/1801.01401
[24] Towards Accurate Generative Models of Video: A New Metric & Challenges. https://arxiv.org/abs/1812.01717
[25] Quo Vadis, Action Recognition? A New Model and the Kinetics Dataset. https://arxiv.org/abs/1705.07750
[26] Rethinking the Inception Architecture for Computer Vision. https://arxiv.org/abs/1512.00567
[27] Motionlet: Towards Controllable Video Generation and Editing with Multimodal Conditions. https://arxiv.org/abs/2401.01827
[28] Progressive Distillation for Fast Sampling of Diffusion Models. https://arxiv.org/abs/2202.00512
[29] Schedules. https://huggingface.co/docs/diffusers/v0.33.0/en/api/schedulers
[30] Photorealistic Text-to-Image Diffusion Models with Deep Language Understanding. https://arxiv.org/abs/2205.11487
[31] Dreamvideo: Customizing Text-to-Video Generation with Multiple Subjects. https://arxiv.org/abs/2401.09962
[32] Control-A-Video: Controllable Text-to-Video Generation with Diffusion Models. https://controlavideo.github.io/
[33] Introducing Stable Cascade. https://stability.ai/news/introducing-stable-cascade

Mark as Complete

Partner With Us
Be an Affiliate
Become a Contributor

Company & Legal
Our Team
Newsletter
Privacy Policy
Terms of Service
Do Not Sell or Share My Personal Information

Support
hi@bytebytego.com
Report a Bug

Resources
YouTube
Visual Dev Guides
Find Jobs
Prepare for Coding Interviews

Ask Anna

Copyright ©2022-2026 BytebyteGo Inc. All rights reserved.